3. Capa de transporte

Grado en Ingeniería Informática y Tecnologías Virtuales
Grado en Ingeniería del Software

Software de Comunicaciones

Curso 2025-2026

Universidad LOYOLA

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

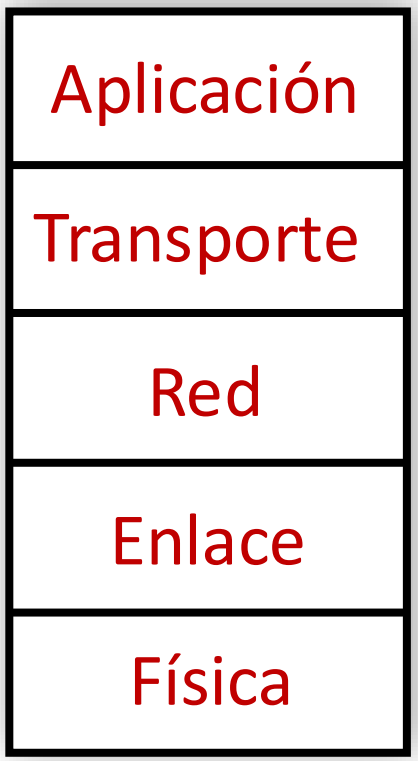
d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

Arquitectura: modelo de capas

- *Aplicación:* aplicaciones de red
  - HTTP, IMAP, SMTP, DNS, FTP, NTP, DHCP
- *Transporte:* transferencia de datos entre procesos
  - **TCP, UDP**
- *Red:* enrutado de datagramas desde la fuente al destino
  - IP, ARP, ICMP, IGMP
- *Enlace:* transferencia de datos entre vecinos
  - Ethernet, 802.11 (WiFi), PPP
- *Física:* bits en el medio físico



Protocolos de la capa de transporte

UDP (*User Datagram Protocol*)

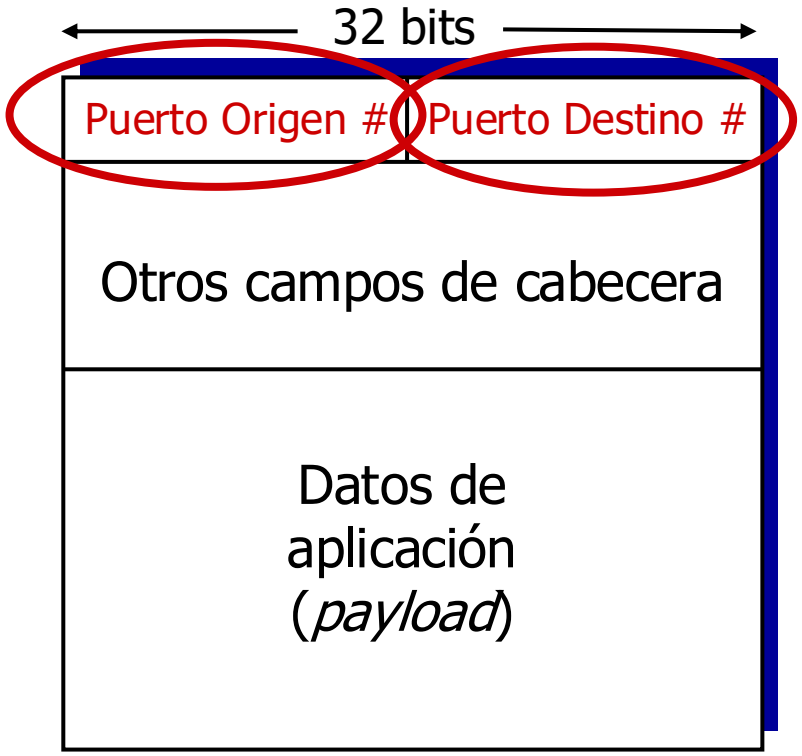
- Proporciona una transferencia **no fiable** de grupos de bytes (datagramas) entre cliente y servidor.
- No recupera errores en la transmisión.
- Sólo extiende el servicio ofrecido por la capa IP a los programas (procesos). IP solo envía información de un host a otro.
- Los datos transmitidos pueden ser perdidos o entregados desordenados.

TCP (*Transmission Control Protocol*)

- **Fiable**, orientada a un flujo de bytes.
- Detecta y recupera errores, ordenación y pérdida de paquetes.
- Proporciona un **canal seguro** de envío de bytes, sin que la capa de aplicación tenga que preocuparse de estos problemas.

Segmento TCP/UDP

- *Hosts* reciben datagramas IP con dirección IP fuente y destino.
- Cada datagrama lleva un segmento de la capa de transporte.
- Cada segmento tiene la información del número de puerto origen y destino.
- *Hosts* usan dirección IP y número de puertos para direccionar segmentos al socket apropiado.
- Puertos entre 0 y 1023 están restringidos (reservados para aplicaciones conocidas).



Formato segmento TCP/UDP

Puertos

Puerto	Protocolo	Uso
20, 21	FTP	Transferencia archivos
22	SSH	Login remoto
25	SMTP	Email
80	HTTP	WWW
110	POP-3	Acceso email remoto
143	IMAP	Acceso email remoto
443	HTTPS	Web segura
543	RTPS	Media Player Control
631	IPP	Impresora compartida

Lista de puertos reservados: RFC 1700
Actualizado en ***Internet Assigned Numbers Authority***: www.iana.org (RFC 3232)

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

UDP

INTERNET STANDARD

RFC 768

J. Postel
ISI
28 August 1980

User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format

```
 0      7 8     15 16    23 24    31
+--------+--------+--------+--------+
|     Source      |   Destination   |
|      Port       |      Port       |
+--------+--------+--------+--------+
|                 |                 |
|     Length      |    Checksum     |
+--------+--------+--------+--------+
|
|          data octets ...
+---------------- ...
```

UDP

¿Por qué usar UDP si TCP proporciona un servicio de transferencia fiable de datos?

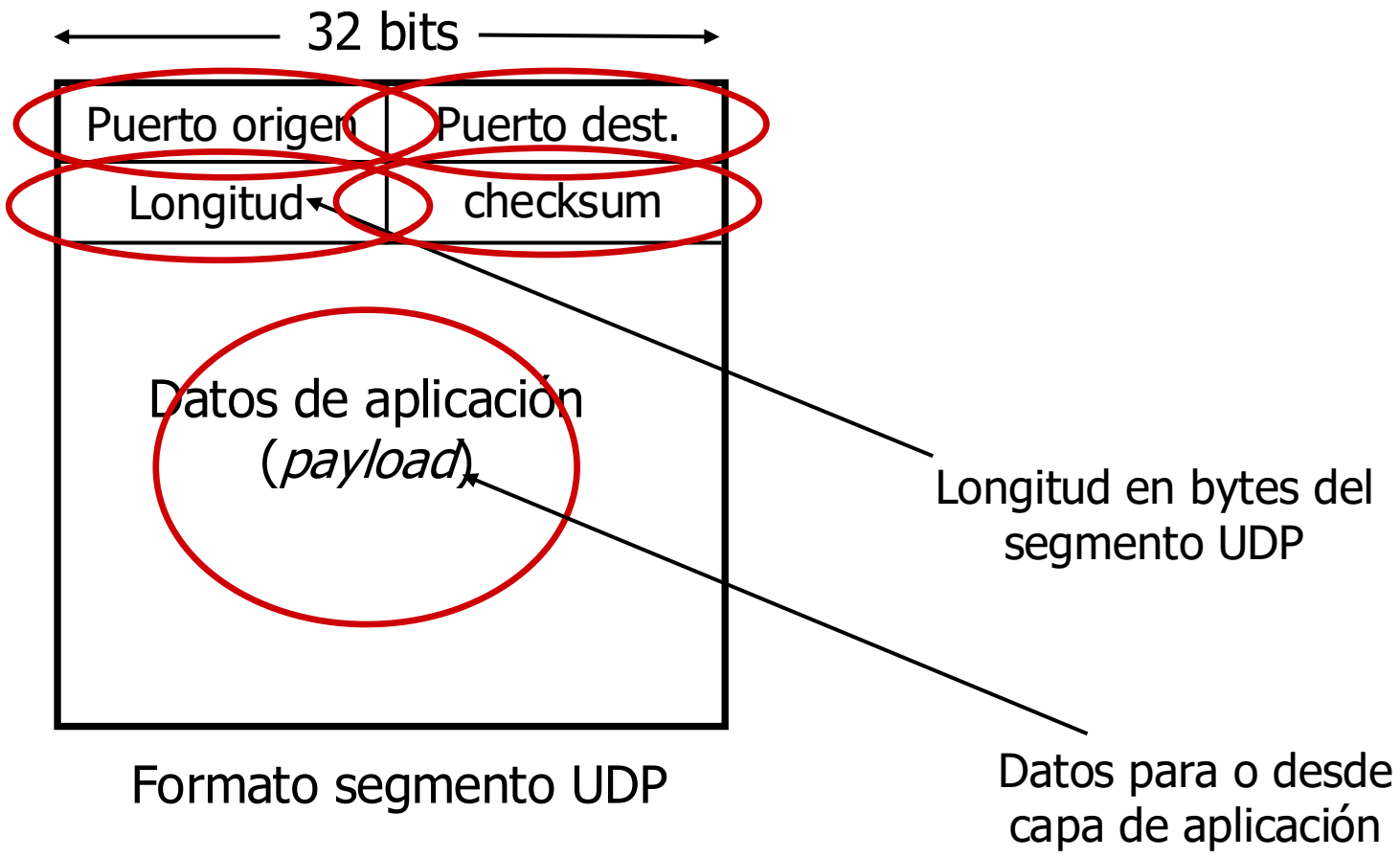
- Ciertas aplicaciones se adaptan mejor a UDP.
- **Control más fino** en el **nivel de aplicación** sobre qué es enviado y cuando.
  - UDP empaqueta directamente los datos en un segmento UDP y lo pasa inmediatamente a la capa de red. Por el contrario, TCP implementa un mecanismo de control de congestión. TCP reenviará segmentos hasta que no sean asentidos por el destinatario.
  - Por ejemplo, algunas aplicaciones en tiempo real pueden tolerar alguna pérdida de datos.
- No hay establecimiento de la conexión. TCP establece un protocolo de establecimiento de conexión antes de enviar datos. UDP envía datos sin ningún tipo de conexión preliminar. Por tanto, **UDP no establece ningún retraso en el establecimiento de la conexión**.
  - Esta es la razón por qué DNS corre sobre UDP y no sobre TCP. DNS sería más lento si fuese sobre TCP.

UDP

¿Por qué usar UDP si TCP proporciona un servicio de transferencia fiable de datos?

- No estado de la conexión. TCP mantiene un estado de la conexión en el sistema final, incluyendo buffers de envío y recepción, parámetros de control de congestión y parámetros del número de asentimientos y secuencias. UDP no tiene en cuenta estos parámetros.
  - Un servidor en general puede soportar más clientes activos cuando va sobre UDP.
- Pequeña **cabecera** de paquetes. El segmento **TCP** tiene **20 bytes** de cabecera en cada segmento, mientras que **UDP** tiene sólo **8 bytes**.
- También es posible tener una transferencia fiable de datos cuando usamos UDP, pero este mecanismo se tiene que implementar en la capa de aplicación.

UDP: estructura del segmento



Formato segmento UDP

UDP

UDP checksum

El objetivo es detectar si los bits del segmento UDP han sido alterados. Detección de errores.

- El remitente trata el contenido del segmento UDP como una secuencia de enteros de 16 bits.
- *Checksum*: suma del contenido del segmento (complemento a 1).
- El resultado es puesto en el campo *checksum*.
- El receptor computa el segmento recibido.
- Comprueba si el *checksum* es igual al valor del campo *checksum*.
  - No es igual: error detectado.
  - Igual: No error detectado (No detectado no quiere decir que no haya).

UDP

UDP checksum

	Número 1	Número 2	Suma
Transmitido:	5	6	11
Recibido:	4	6	11

Suma calculada en el receptor

Suma calculada en el remitente

UDP

UDP checksum

	Número 1	Número 2	Suma
Transmitido:	5	6	11
Recibido:	4	6	11

Suma calculada en el receptor ≠ **Suma calculada en el remitente**

¡ERROR DETECTADO!

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

TCP: un poco de historia

Vinton Cerf, Robert Kahn y TCP/IP.

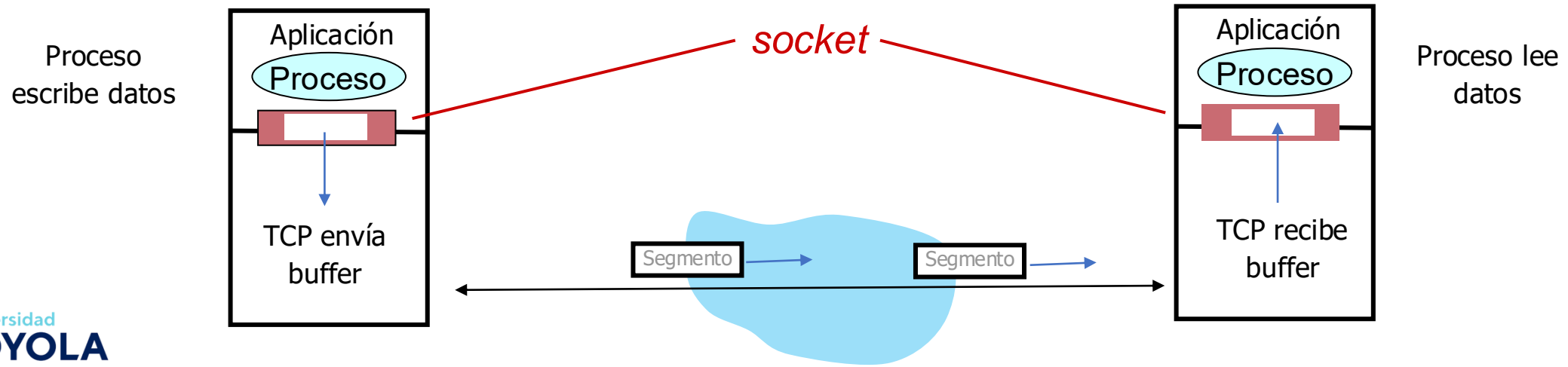
- En los años 70, las redes de conmutación de paquetes empezaron a proliferar. ARPAnet era una de esas redes. Cada una de esas redes tenían sus propios protocolos.
- Necesidad de interconectar esas redes.
- Inventaron un protocolo llamado TCP/IP para interconectar esas redes (lo veían como una sola entidad).
- Artículo sobre TCP/IP publicado en Mayo de 1974 en *IEEE Transactions on Communications Technology*.
- Separados después en dos protocolos, TCP e IP.
- Diseñados antes de la aparición de ordenadores personales, smartphones, tablets, Ethernet, WiFi, web, social media…

TCP

- TCP definido en RFCs: 793, 1122, 2018, 5681, 7323
- Conexión punto a punto, un remitente y un receptor.
- Conexión fiable.
- Proporciona un servicio ***full-duplex***: flujo de datos bidireccional en la misma conexión.
- Orientado a conexión (intercambio de mensajes de control).
- Asentimientos.
- **Control de flujo** (el remitente no agobia al receptor) y **de congestión**.

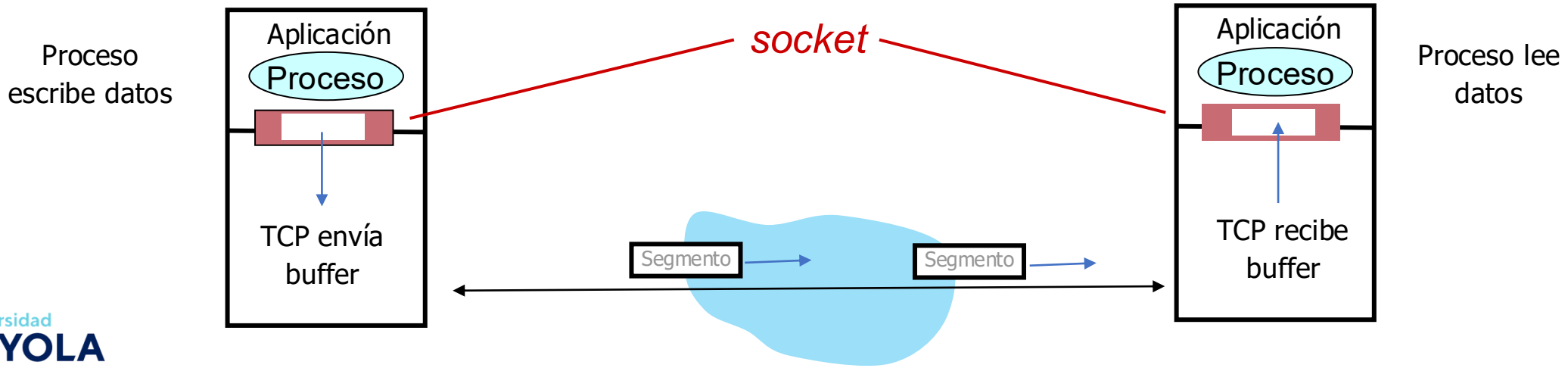
TCP

- El proceso cliente pasa un flujo de datos a través del socket. Los datos están **en manos de TCP** que corre en el cliente.
- De vez en cuando TCP coge trozos de datos del buffer de envío y lo pasa a la capa de red.
- La máxima cantidad de datos que puede ser cogida y ubicada en un segmento está limitada por **MSS** (*Maximum Segment Size*).
- MSS es típicamente establecido primero determinando la trama más grande de la capa de enlace que puede ser enviada por el equipo local (la llamada MTU, *Maximum Transmission Unit*) y asegurando que un segmento TCP (cuando es encapsulado en un datagrama IP) más la longitud de la cabecera TCP/IP (típicamente 40 bytes) encaja en una única trama de la capa de enlace. Por ejemplo, Ethernet tiene una MTU de 1500 bytes. Un valor típico de MSS es 1460 bytes.

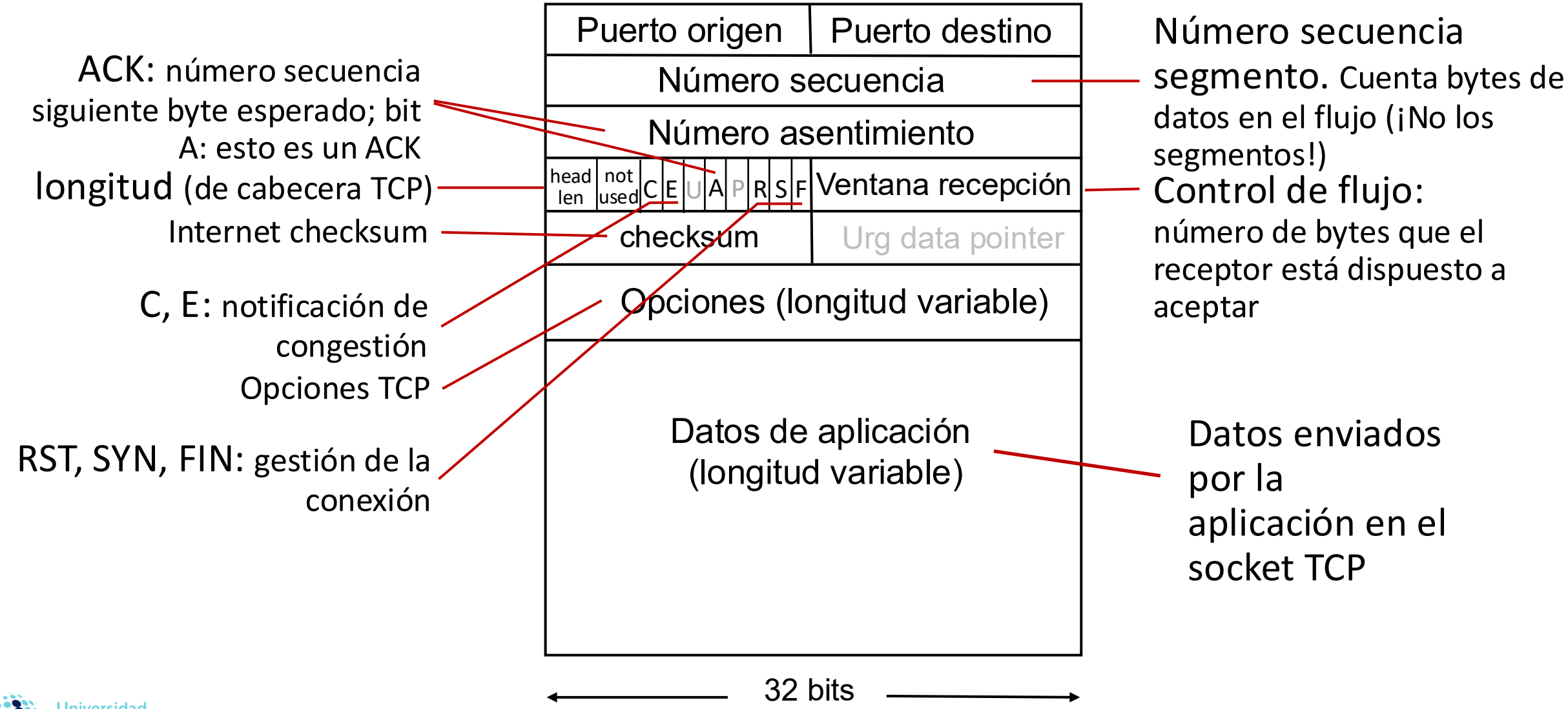


TCP

- **MSS es la cantidad máxima de datos de la capa de aplicación en el segmento** (no el segmento total incluyendo la cabecera).
- TCP empareja cada trozo de datos con una **cabecera TCP**, formando los segmentos TCP.
- Los segmentos son pasados a la capa de red y encapsulados en un **datagrama IP**.
- Cuando TCP recibe un segmento, los datos son ubicados en el buffer de recepción. La aplicación lee el flujo de datos de este buffer.



TCP: estructura del segmento



Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

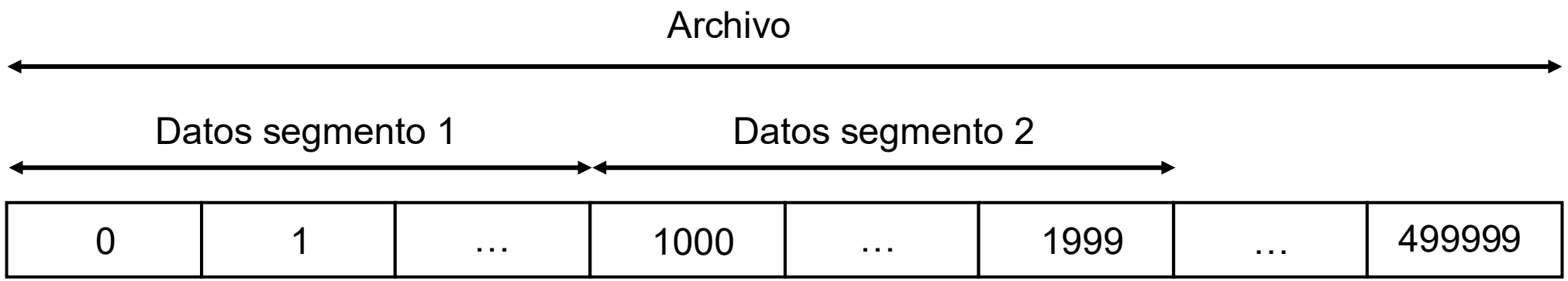
d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

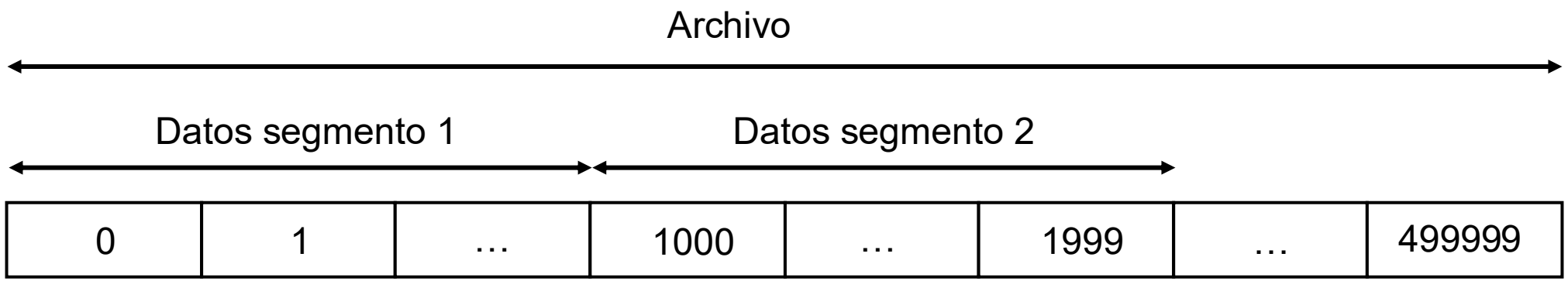
TCP: número de secuencia y asentimiento

- Usados para la transferencia fiable de datos.
- Proceso en equipo A quiere enviar un flujo de datos a proceso en equipo B.
- TCP en el equipo A numerará cada byte en el flujo de datos.
- Ejemplo: Flujo de datos de 500000 bytes. MSS es 1000 bytes y que el primer byte del flujo es 0.
- TCP construye 500 segmentos. El **número de secuencia** del primer segmento es 0, el segundo segmento tiene un número de secuencia 1000, el tercero 2000 … Cada número es añadido en el campo de la cabecera.



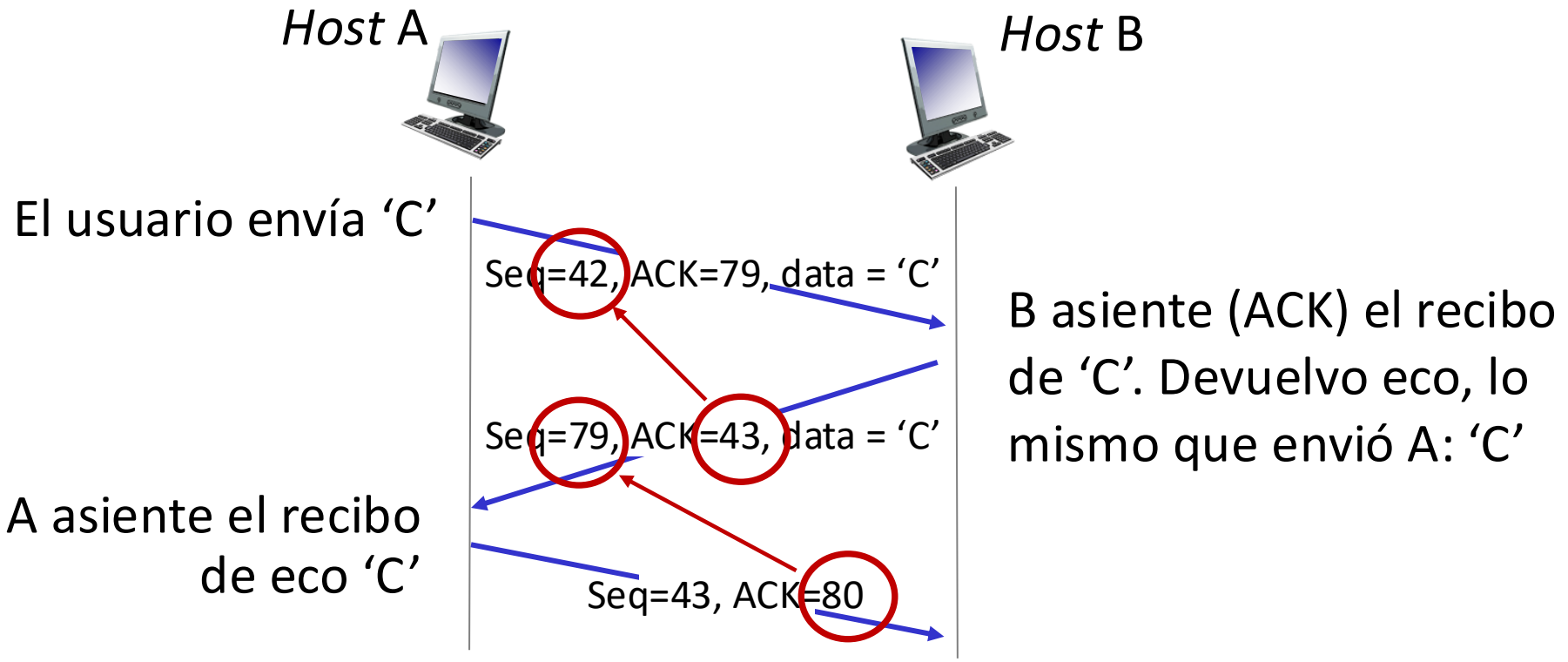
TCP: número de secuencia y asentimiento

- El número de asentimiento que el equipo A pone en el segmento es el número de secuencia del siguiente byte que el equipo A espera del equipo B.
- Ejemplo I: imagina que el equipo A ha recibido todos los bytes numerados del 0 al 535 del equipo B y suponemos que está a punto de enviar un segmento al equipo B. Equipo A está esperando a segmento 536 y todos los bytes posteriores. Entonces equipo A pone 536 en el segmento a enviar a B.
- Ejemplo II: imagina que el equipo A ha recibido todos los bytes numerados del 0 al 535 del equipo B y un segmento que contiene los bytes 900 a 1000. Por alguna razón equipo A aún no ha recibido los bytes 536 a 899. Entonces equipo A está esperando a byte 536. El siguiente segmento a B contendrá 536 en el campo número de asentimiento.



Ejemplo número de secuencia y asentimiento

- **Telnet** (RFC 854), es un protocolo de la capa de aplicación usado para *login* remoto. Es preferible usar el protocolo **SSH**, ya que los datos en Telnet no están encriptados (incluidos la clave) por lo que tiene serios problemas de seguridad.
- Usaremos Telnet como ejemplo para ver número de secuencia y asentimiento.



Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

TCP: *Round Trip Time* (RTT) y *timeout*

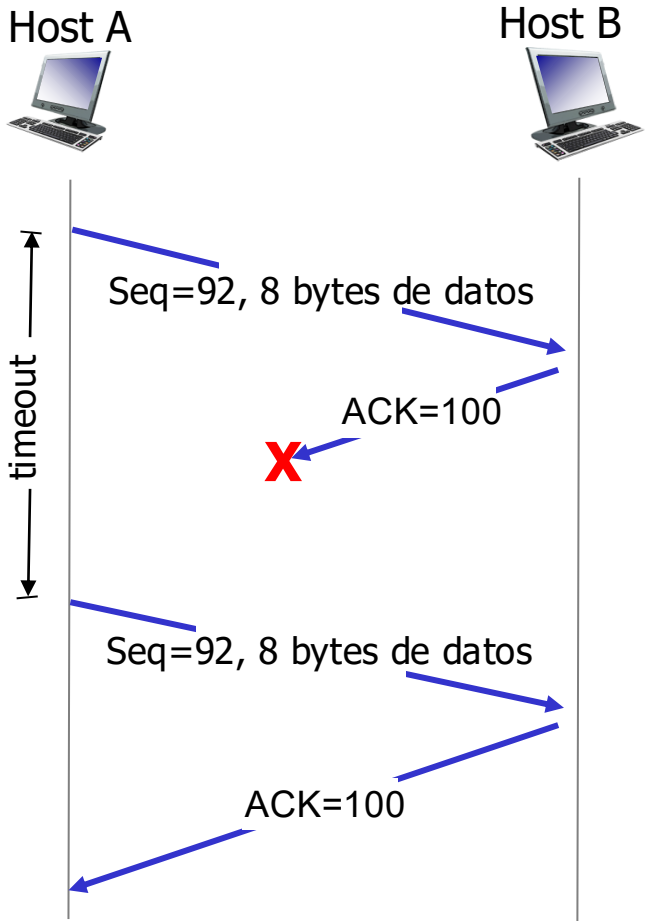
- TCP usa un mecanismo de retransmisión (*timeout*) para recuperar los segmentos perdidos.
- La pregunta es cuál debe ser la longitud del intervalo de *timeout*.
- ***Timeout* debe ser mayor que RTT** para evitar retransmisiones innecesarias. Pero si es demasiado largo tenemos una reacción lenta.

Receptor TCP: Generación de asentimientos (RFC 5681)

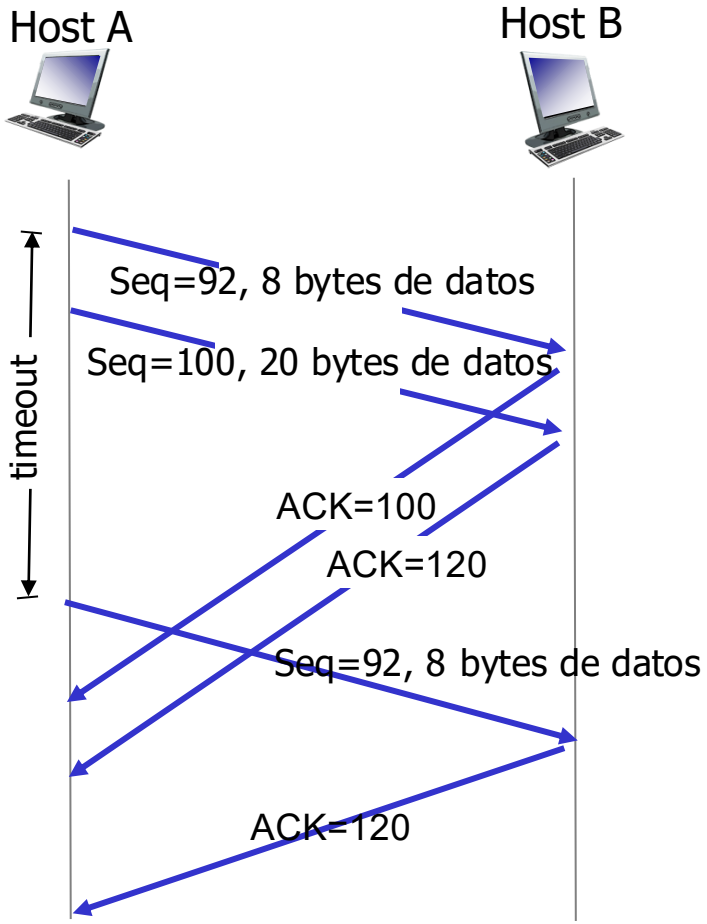
Evento en el receptor	Acción en el receptor
Llegada de segmento en orden con número de secuencia esperado. Todos los datos hasta el esperado número de secuencia son asentidos.	ACK retrasado. Espera hasta 500 ms para el siguiente segmento. Si no hay siguiente segmento, se envía el ACK.
Llegada de un segmento en orden con número de secuencia esperado. Otro segmento tiene un ACK pendiente.	Inmediatamente enviar único ACK acumulativo, asintiendo ambos segmentos en orden.
Llegada de un segmento fuera de orden con un número de secuencia más alto del esperado. Detección de falta de datos.	Inmediatamente enviar ACK duplicado, indicando el número de secuencia del próximo byte esperado.
Llegada de un segmento que parcialmente o completamente llena el hueco en los datos recibidos.	Inmediatamente enviar ACK, proporcionando el comienzo del segmento al inicio del hueco.

Nota: Un ACK duplicado es aquel que asiente un segmento del que el receptor ya ha recibido un asentimiento anterior.

TCP: escenarios de retransmisión

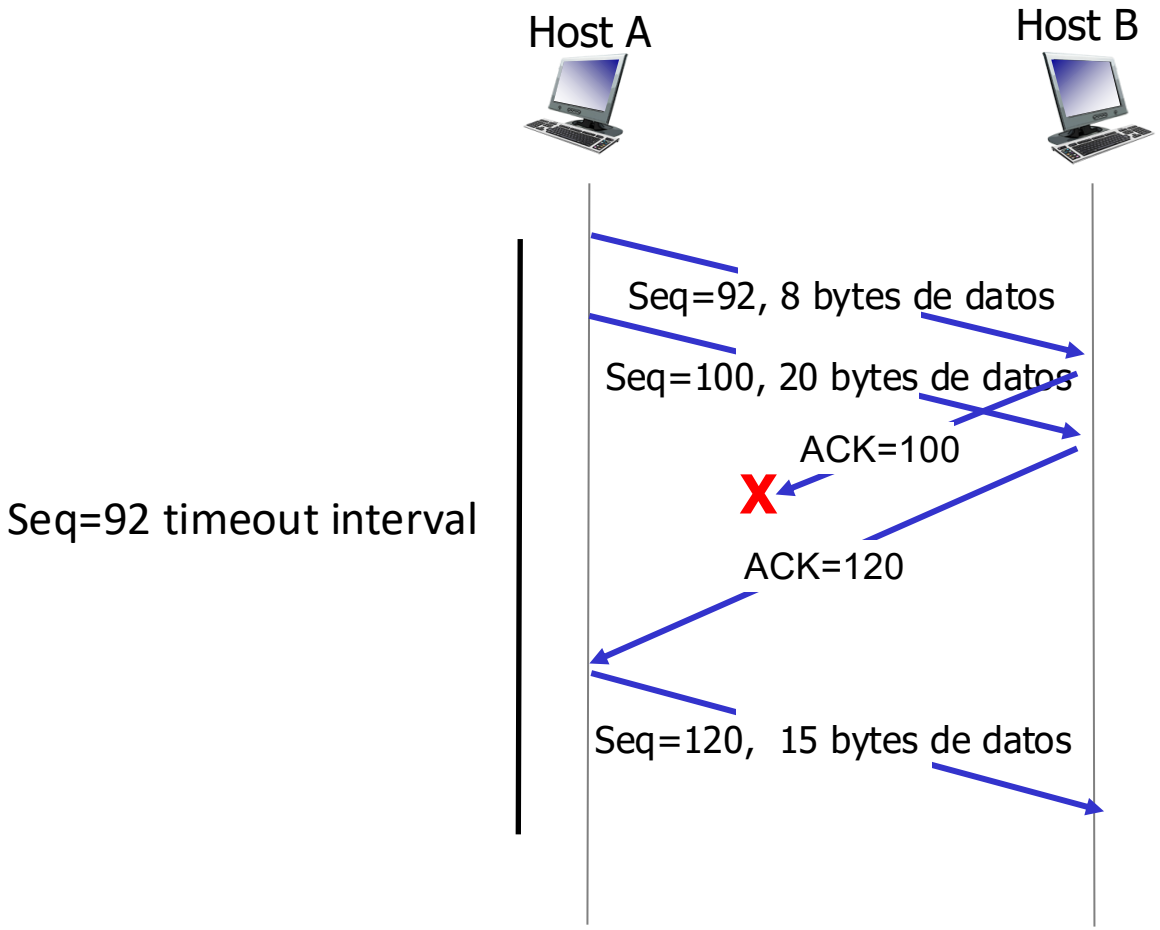


ACK perdido



Timeout prematuro

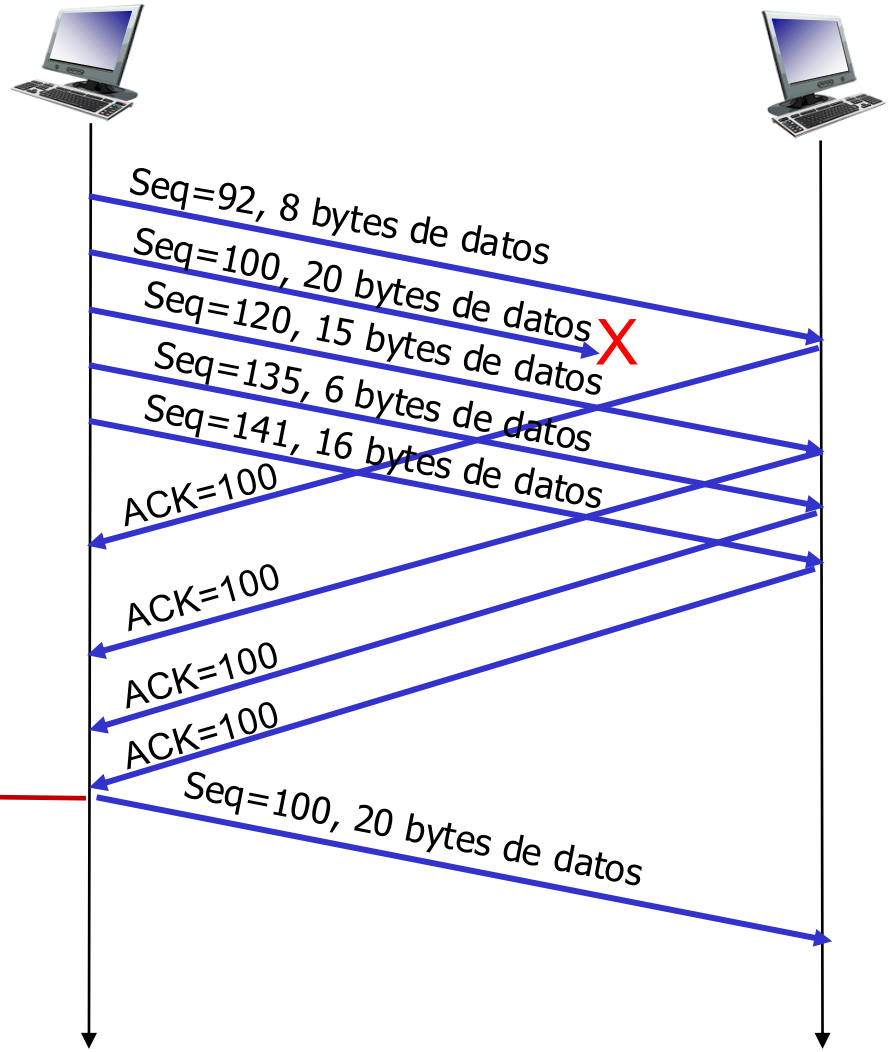
TCP: escenarios de retransmisión



ACK acumulativo protege por una pérdida temprana del ACK. Evita la retransmisión del segmento anterior.

TCP: retransmisión rápida

Si el remitente envía 3 ACKs para los mismos datos, se reenvía el segmento no asentido con el número de secuencia menor (Probablemente ese segmento no asentido se ha perdido así que no esperamos al *timeout* para retransmitir).



El recibo de 3 ACKs duplicados indica 3 segmentos recibidos después de un segmento perdido. Segmento perdido es probable, por tanto, se retransmite.

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

d. Control de flujo

e. Gestión de la conexión

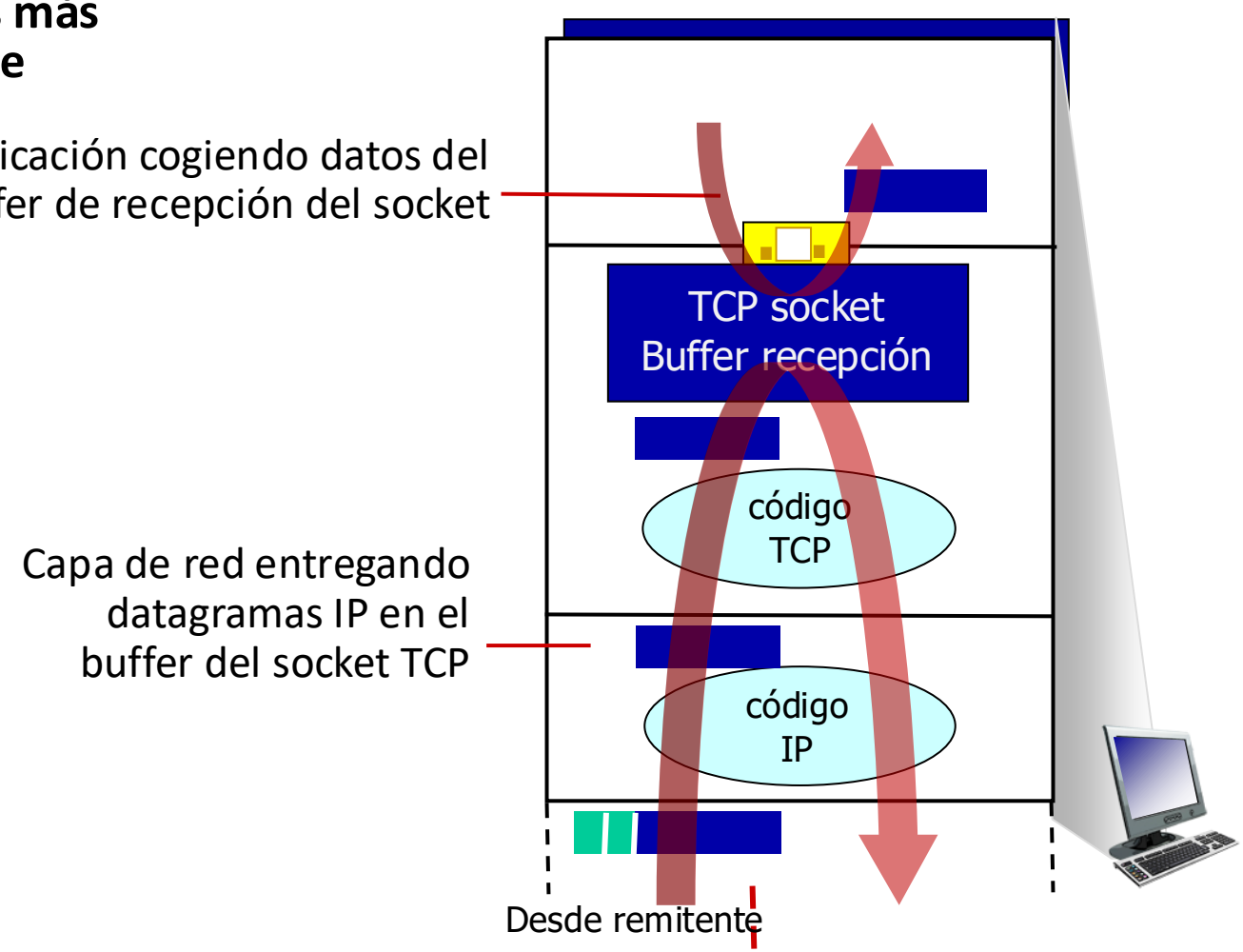
f. Control de congestión

TCP: Control de flujo

¿Qué ocurre si la capa de red entrega los datos más rápido que la lectura de los datos por la capa de aplicación?

TCP: Control de flujo

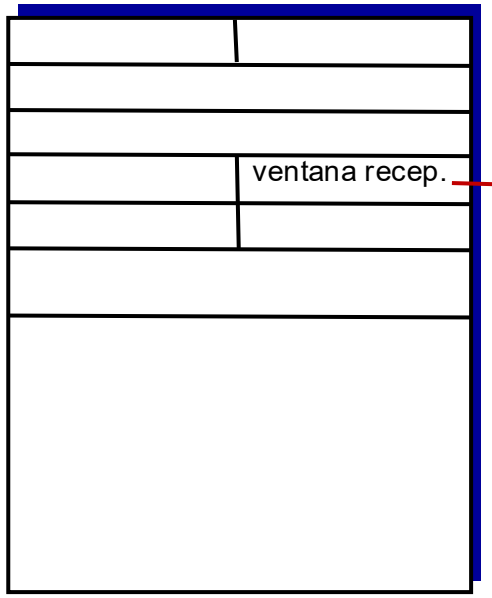
¿Qué ocurre si la capa de red entrega los datos más rápido que la lectura de los datos por la capa de aplicación?



Torre de protocolos del receptor

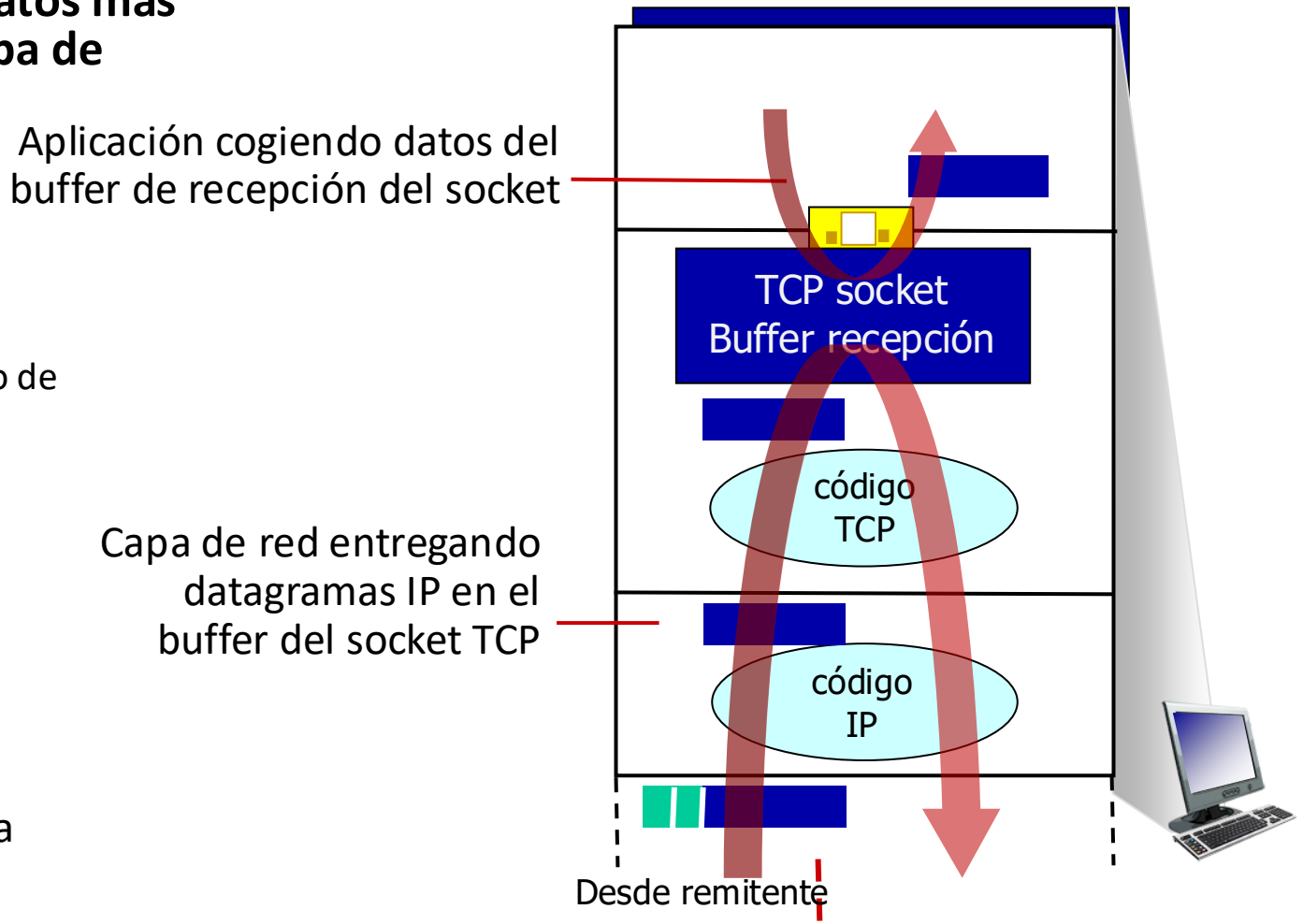
TCP: Control de flujo

¿Qué ocurre si la capa de red entrega los datos más rápido que la lectura de los datos por la capa de aplicación?



Control de flujo: número de bytes que el receptor está dispuesto a aceptar

El receptor controla al remitente, de forma que el remitente no satura el buffer del receptor transmitiendo demasiado rápido.



Torre de protocolos del receptor

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

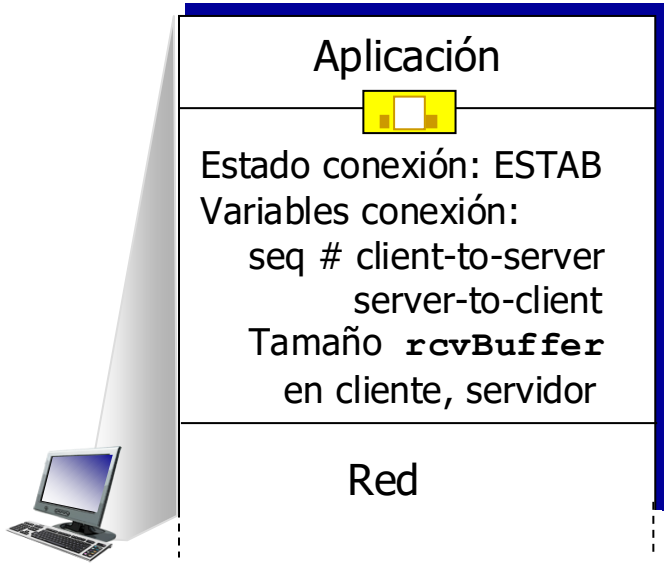
d. Control de flujo

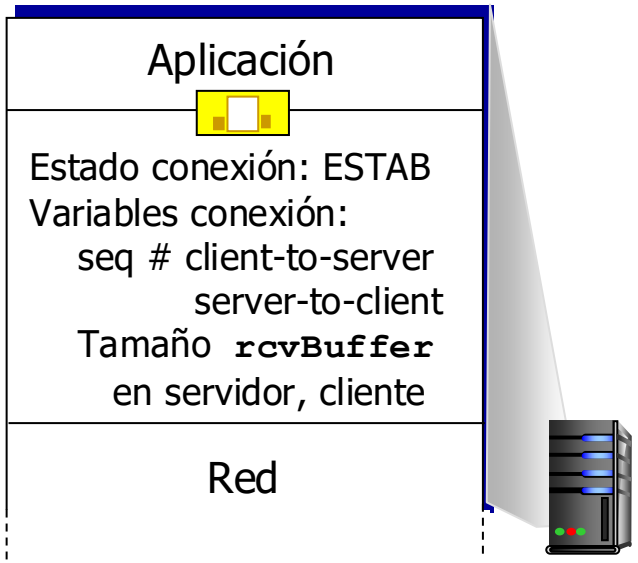
e. Gestión de la conexión

f. Control de congestión

TCP: gestión de la conexión

- Antes de intercambiar datos, remitente/receptor "*handshake*".
- Se ponen de acuerdo para establecer conexión (cada uno conoce la disponibilidad del otro para establecer conexión).
- Acuerdo en parámetros de la conexión (números de secuencia iniciales, …).






```
Socket clientSocket =
  newSocket("hostname","port number");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```

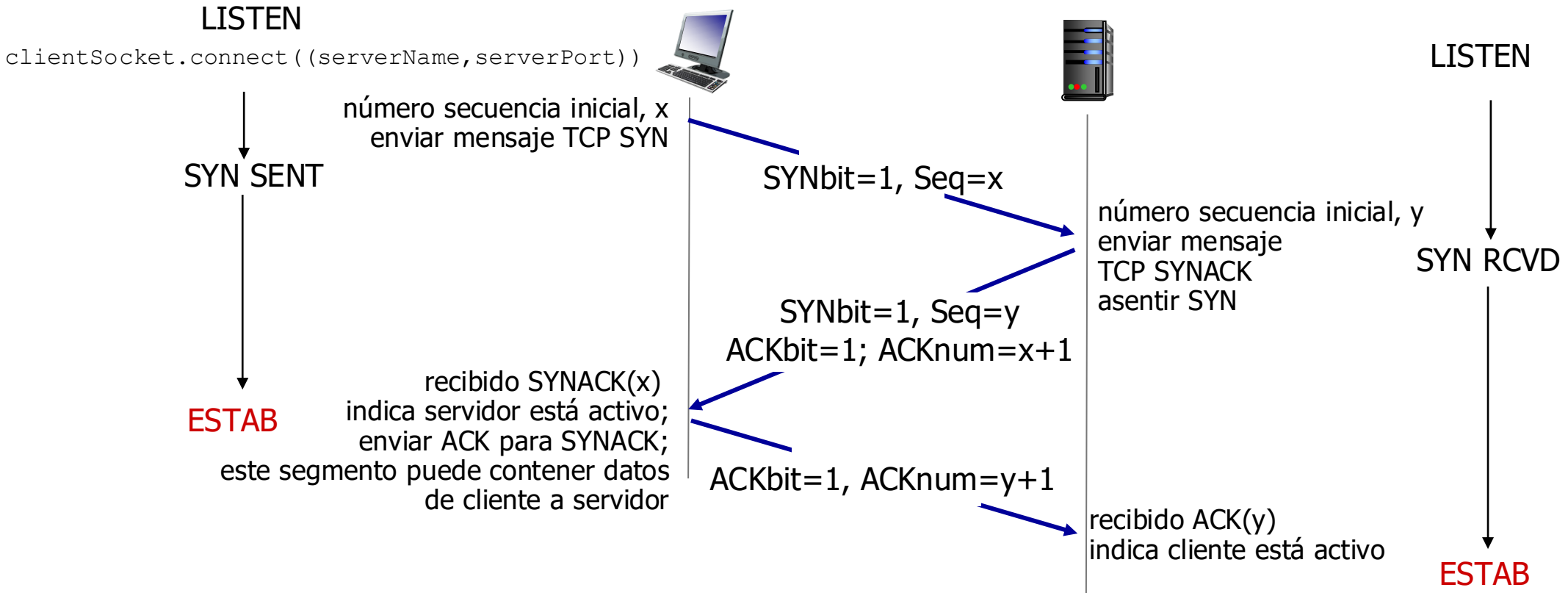
TCP: establecimiento de la conexión

Estado servidor

```
serverSocket = socket(AF_INET,SOCK_STREAM)
serverSocket.bind(('',serverPort))
serverSocket.listen(1)
connectionSocket, addr = serverSocket.accept()
```

Estado cliente

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```



3-way handshake

TCP: cerrando la conexión

- El cliente y el servidor cierran su lado de la conexión (envían segmento TCP con bit FIN igual a 1).
- Responden al mensaje FIN con un ACK (al recibir FIN, ACK puede ser combinado con su propio mensaje FIN).
- Intercambios simultáneos de FIN pueden ser manejados.

Contenido

1- Protocolos de la capa de transporte

2- UDP

3- TCP

a. Introducción

b. Número de secuencia y asentimiento

c. Mecanismos de retransmisión

d. Control de flujo

e. Gestión de la conexión

f. Control de congestión

TCP: Control de congestión

- Muchas fuentes enviando datos demasiado rápido de forma que la red no puede manejar.
- Consecuencias:
  - Largos retrasos (buffers en las colas de los *routers*).
  - Pérdida de paquetes.
- Control de flujo y control de congestión son diferentes.

Control de congestión: muchos remitentes enviando demasiado rápido

Control de flujo: un remitente enviando demasiado rápido para un receptor

TCP: Control de congestión

- El enfoque de TCP es limitar la tasa de envío de cada remitente en función de la congestión de red percibida.
- Si el remitente TCP percibe que hay poca congestión en la ruta entre su destinatario y él, entonces incrementa su tasa de envío.
- Si el remitente TCP percibe que hay congestión en su ruta, entonces reduce su tasa de envío.
- Con este enfoque aparecen tres preguntas:
  - ¿Cómo el remitente TCP limita la tasa de transmisión a su destino?
  - ¿Cómo el remitente TCP percibe que hay congestión?
  - ¿Qué algoritmo debe usar el remitente para cambiar su tasa de envío en función de la congestión percibida en la conexión?

TCP: Control de congestión

- ¿Cómo el remitente TCP limita la tasa de transmisión a su destino?
  - Mediante la ventana de congestión
- ¿Cómo el remitente TCP percibe que hay congestión?
  - Pérdida por timeout. No llega ningún ACK en tiempo esperado.
  - ACKs duplicados. El receptor sigue pidiendo el mismo paquete. El paquete se perdió, pero la red funciona
- ¿Qué algoritmo debe usar el remitente para cambiar su tasa de envío en función de la congestión percibida en la conexión?
  - Algoritmo AIMD (*Additive Increase Multiplicative Decrease*)

Universidad
LOYOLA